Relatório técnico — CardioIA Fase 6

Sistema preditivo multiagente para análise de risco em ambiente hospitalar simulado

Autor	Renan de Oliveira Mendes (RM563145)
Curso	FIAP — Inteligência Artificial / Data Science
Entrega	Fase 6 · Enterprise Challenge
Data	17/10/2026
Repositório	https://github.com/ReMendess/cardioia_Fase6

1. Problema e objetivo

Ambientes de urgência precisam **priorizar** pacientes diante de recursos limitados. Na versão acadêmica proposta, a decisão de "pico de risco" deve ser: (i) **calculada** de forma reprodutível, (ii) **explicada** com as regras que a originaram e (iii) **organizada** em uma resposta legível.

O objetivo desta entrega é construir um sistema que:

1. treine um classificador supervisionado para a variável `pico_risco` sobre a base sintética do material da disciplina;
2. aplique as **10 regras determinísticas** da base de conhecimento oficial;
3. use **quatro agentes** orquestrados pelo OpenAI Agents SDK (*handoffs* reais) para conduzir o caso, mantendo **fora do modelo de linguagem** qualquer cálculo crítico;
4. devolva um **contrato de saída validado** (Pydantic), consumível em texto ou JSON.

2. Dados

Item	Descrição
Origem	`data/base_dados.py` (material oficial da aula)
Natureza	**100% sintética**, gerada com `np.random.seed(42)`
Registros	5.000
Variáveis de entrada	10 (idade, carga do sistema, tempo de espera, FC, SpO_2, PA sistólica, temperatura, densidade de eventos, eventos em 24 h, disponibilidade de recursos)
Alvo	`pico_risco` ∈ {0, 1}, com **1.329 positivos (26,58%)**
Rotulagem	regra determinística: `risk_score ≥ 2`, onde `risk_score` soma 6 indicadores booleanos
Integridade	SHA-256 da base registrado no metadata do modelo

Cuidado metodológico implementado: `risk_score` **não** é usado como variável de entrada (seria vazamento direto do alvo). A verificação existe no notebook e em `dataset.separar_features_e_alvo`.

3. Metodologia de Machine Learning

Decisão	Adotada
Features	as 10 colunas originais, na ordem do arquivo oficial
Divisão	80/20 **estratificada**, `random_state = 42` → 4.000 treino / 1.000 teste
Modelos comparados	`LogisticRegression` (pipeline com `StandardScaler`) e `RandomForestClassifier(n_estimators=300, random_state=42)`
Mesmos conjuntos	ambos treinados e avaliados sobre **o mesmo** treino e o mesmo teste
Critério de seleção	declarado **antes** de medir: maior *recall* da classe 1; empate por acurácia; depois interpretabilidade
Uso do teste	**uma única vez**, apenas para medir — sem ajuste de hiperparâmetros
Limiar de decisão	0,5 — **operacional**, não calibrado clinicamente
Métricas	acurácia, *recall*, precisão, F1, matriz de confusão e relatório de classificação

4. Resultados

4.1 Desempenho no conjunto de teste (mesmo para os dois modelos)

Modelo	Acurácia	Recall (1)	Precisão (1)	F1 (1)	Matriz de confusão `[[TN, FP], [FN, TP]]`
Regressão Logística	0,8160	0,5526	0,6934	0,6151	`[[669, 65], [119, 147]]`
Random Forest (escolhido)	**0,9990**	**0,9962**	1,0000	0,9981	`[[734, 0], [1, 265]]`
Regra determinística (teto)	1,0000	1,0000	1,0000	1,0000	`[[734, 0], [0, 266]]`

O critério pré-declarado (**maior recall da classe 1**) selecionou o **Random Forest**. O único falso negativo tem `risk_score = 2`, isto é, está exatamente na fronteira do corte do rótulo.

4.2 Leitura crítica (o que os números **não** significam)

- O alvo é uma **função determinística** de 6 variáveis com limiares fixos; um modelo flexível **reconstrói a regra**. A acurácia de 0,999 mede *aderência à regra*, não generalização.
- A diferença para a Regressão Logística (0,816) indica que a fronteira de decisão **não é linearmente separável** nessas variáveis — a vantagem do Random Forest vem da forma do rótulo.
- O "teto" de 1,0 da regra determinística é **tautológico** (a regra define o próprio alvo) e serve apenas como referência de sanidade.
- **Nenhuma conclusão clínica** é possível: dados sintéticos, sem pacientes reais, sem validação externa e com limiar não calibrado.

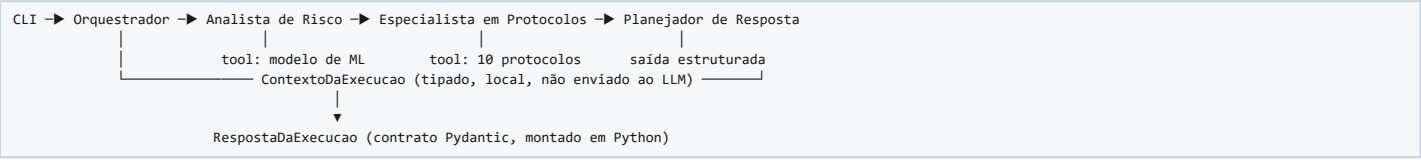
4.3 Verificação de software

Evidência	Resultado
Suíte automatizada (`python -m pytest tests -q`)	**281 testes aprovados, 3 pulados** (20 s, sem usar a API)
Pipeline com o SDK real e modelo simulado	Handoffs, tools, contexto tipado e contrato exercitados

Evidência	Resultado
Execução completa com LLM real	❌ **não ocorreu** — a conta respondeu `HTTP 429 / insufficient_quota`
Reprodutibilidade do artefato	`.pkl` com SHA-256 idêntico antes/depois da correção de caminho
Portabilidade	metadata sem caminho absoluto; testes executam de outros diretórios

Declaração honesta de escopo: a *condução dos handoffs por um LLM real* **não está comprovada** nesta entrega. O que está comprovado é o encadeamento determinístico (SDK + tools + contrato) e que uma falha de API **não** é mascarada como sucesso.

5. Arquitetura multiagente



Agente	Responsabilidade	Não faz
Orquestrador	valida entradas, exige o artefato, inicia o run, monta/valida o contrato	não responde por conta própria
Analista de Risco	chama a tool que consulta o modelo salvo	não estima, não arredonda, não treina
Especialista em Protocolos	chama a tool que aplica as 10 regras oficiais	não cria regra nem altera limiar
Planejador de Resposta	consolida narrativa, justificativas e pontos de atenção	não recalcula nem introduz números

Princípio central de confiança. Probabilidade, classe, protocolos e justificativas são calculados por **Python** e gravados em um contexto tipado (`RunContextWrapper.context`), que a documentação do SDK define como objeto **local, não enviado ao modelo**. As *tools* não aceitam argumentos: o LLM decide **quando** consultar, nunca **com que números**. A camada puramente determinística (dataset, modelo, protocolos, contrato) é testável sem rede e sem LLM.

Os *handoffs* são **unidirecionais** (pipeline com ordem fixa), o `max_turns` limita o run (padrão 10) e o *tracing* do SDK está **desativado**. Detalhes, diagramas e tratamento de falhas: `arquitetura.md`.

6. Camada determinística de protocolos

As 10 regras de `data/base_conhecimento.py` são vinculadas **por posição validada** a condições numéricas em `src/cardioia/knowledge/protocols.py` — não por busca textual aproximada. Garantias cobertas por teste:

- operadores **estritos** (ex.: `spo2_media == 88` **não** dispara `spo2_media < 88`);
- contrato falha explicitamente se o texto oficial mudar (evita casar a regra errada);
- apenas protocolos satisfeitos são retornados, sem duplicatas;
- cada protocolo retornado inclui condição satisfeita, **valor observado** e **origem** do valor;
- um teste compara os limiares implementados com os extraídos do próprio arquivo oficial.

Os limiares dos protocolos **diferem** dos limiares do rótulo (ex.: `carga_sistema > 85` no protocolo, `> 80` no rótulo). São camadas distintas: o modelo aprende o rótulo; os protocolos seguem sendo avaliados deterministicamente.

7. Limitações

1. **Dados sintéticos** — o alvo é uma regra; não há distribuição clínica real.
2. **Sem execução real completa** — a cota da conta de API impediu validar a condução dos handoffs pelo LLM; a falha é registrada como `ERRO`, não como sucesso.
3. **Limiar não calibrado** — 0,5 é operacional; não houve análise de calibração (curva precisão-recall ou custo de falso negativo).
4. **Conflito de dependências documentado** — `openai-agents 0.18.1` exige `pydantic >= 2.12.2`, enquanto `gradio 5.49.1` exige `< 2.12`: restrições mutuamente insatisfazíveis. Alternativas: SDK 0.4.2 ou ambientes separados. **Não** declarado resolvido.
5. **Sem persistência entre execuções** — cada run tem contexto e histórico próprios.
6. **Escopo acadêmico** — o sistema não é dispositivo médico, não produz diagnóstico nem prescrição e não substitui avaliação profissional.

8. Conclusão e trabalhos futuros

A entrega comprova, de ponta a ponta, um fluxo em que **ML determina os números, protocolos determinísticos determinam as ações e o LLM apenas conduz e redige**, com contrato validado e rastreável. O resultado é auditável: cada valor crítico é rastreável à tool de origem, ao artefato do modelo (com SHA-256) e à regra da base oficial.

A leitura honesta dos resultados exige destacar que as métricas altíssimas decorrem da natureza sintética da base e do critério de escolha declarado — não constituem evidência de desempenho clínico.

Trabalhos futuros: calibrar o limiar por custo de falso negativo; executar o modo real com créditos disponíveis e medir a taxa de condução correta dos handoffs; avaliar `output_guardrails` do SDK para a narrativa; e, se houver dados reais com aprovação ética, avaliar validação externa.

9. Referências técnicas

1. OpenAI. *Agents SDK — documentação oficial* (agents, handoffs, tools, context, run config).
2. scikit-learn. *User Guide — métricas de classificação e matriz de confusão*.
3. Pydantic. *Documentação de modelos e validação*.
4. FIAP. *Material da disciplina* — `data/base_dados.py` e `data/base_conhecimento.py`.